Information Retrieval

Lecture 4

Searching

Inverted File

Inverted file:

An inverted file is list of search terms that are organized for **associative look-up**, i.e., to answer the questions:

- In which documents does a specified search term appear?
- Where within each document does each term appear? (There may be several occurrences.)

In a free text search system, the **word list** and the **postings** file together provide an inverted file system. In addition, they contain the data needed to calculate weights and information that is used to display results.

Inverted File -- Basic Concept

Word	*Document*
abacus	3
	19
	22
actor	2
	19
	29
aspen	5
atoll	11
	34

This is called an **index file**, a **word list**, or a **vocabulary file**.

Stop words are removed before building the index.

Inverted List -- Definitions

Posting: Entry in an inverted file system that applies to a single instance of a term within a document, e.g., there are three postings for "abacus":

abacus	3
abacus	19
abacus	22

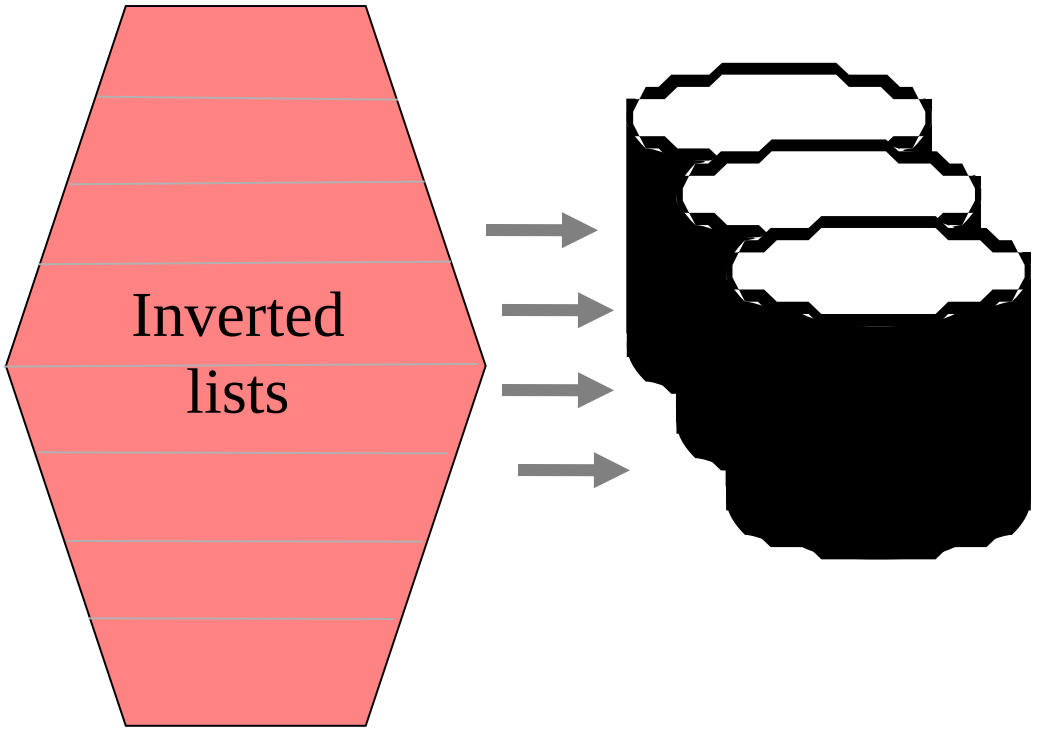
Inverted List: A list of all the postings in an inverted file system that apply to a specific word, e.g.

abacus	3 19 22

Organization of Files for Full Text Searching

Word list (index file) **Postings** **Documents store**

Term	Pointer to postings
ant	
bee	
cat	
dog	
elk	
fox	
gnu	
hog	



Representation of Inverted Files

Document store: Stores the documents. Important for user interface design.

Word list (vocabulary file): Stores list of terms (keywords). Designed for searching and sequential processing, e.g., for range queries, (**lexicographic index**). May be held in memory.

Postings file: Stores an **inverted list** (postings list) of postings for each term. Designed for rapid merging of lists and calculation of similarities. Each list is usually stored sequentially. Can be very large.

Document Store

The **Documents Store** holds the corpus that is being indexed. The corpus may be:

- **primary documents**, e.g., electronic journal articles or Web pages.
- **surrogates**, e.g., catalog records or abstracts, which refer to the primary documents.

Document Store

The storage of the document store may be:

Central - all documents stored together on a single server

Distributed database - all documents managed together but stored on several servers

Highly distributed - documents stored on independently managed servers (e.g., the Web)

Each requires: a **document ID**, which is a unique identifier that can be used by the search system to refer to the document, and a **location counter,** which can be used to specify location of words or characters within a document.

Documents Store for Web Search Systems

For Web search systems:

- A document is a Web page.
- The documents store is the Web.
- The document ID is the URL of the document.

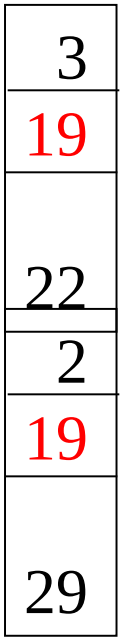
Indexes are built using a **web crawler**, which retrieves each page on the Web for indexing.

Use of Inverted Files for Evaluating a Boolean Query

Examples: abacus *and* actor

Postings for abacus

Postings for actor



To evaluate the *and* operator, merge the two inverted lists with a logical *AND* operation.

Document 19 is the only document that contains both terms, "abacus" and "actor".

Use of Inverted Files for Calculating Similarities

In the term vector space, if **q** is query and $\mathbf{d}_j$ a document, then **q** and $\mathbf{d}_j$ have no terms in common iff $\mathbf{q}.\mathbf{d}_j = 0$.

1. To calculate all the **non-zero similarities** find *R*, the set of all the documents, $\mathbf{d}_j$, that contain **at least one term** in the query:
2. Merge the inverted lists for each term t_i in the query, with a logical *or*, to establish the set, *R*.
3. For each $\mathbf{d}_j$ R, calculate *Similarity*(**q**, $\mathbf{d}_j$), using appropriate weights.
4. Return the elements of *R* in ranked order.

Enhancements to Inverted Files -- Concept

Location: Each posting holds information about the location of each term within the document.

Uses

user interface design -- highlight location of search term

adjacency and *near* operators (in Boolean searching)

Frequency: Each inverted list includes the number of postings for each term.

Uses

term weighting

query processing optimization

Inverted File -- Concept (Enhanced)

Word	*Postings*	*Document*	*Location*
abacus	4	3	94
		19	7
		19	212
		22	56
actor	3	2	66
		19	213
		29	45
aspen	1	5	43
atoll	3	11	3
		11	70
		34	40

Inverted list for term *actor*

Data for Calculating Weights

The calculation of weights requires extra data to be held in the inverted file system.

For each term, t_j and document, $\mathbf{d}_i$

f_{ij} number of occurrences of t_j in $\mathbf{d}_i$

For each term, t_j

n_j number of documents containing t_j

For each document, $\mathbf{d}_i$

m_i maximum frequency of any term in $\mathbf{d}_i$

For the entire document file

n total number of documents

Word List: Individual Records for Each Term

The record for term *j* in the word list contains:

term *j*
pointer to inverted (postings) list for term *j*
number of documents in which term *j* occurs (n_j)

Postings File

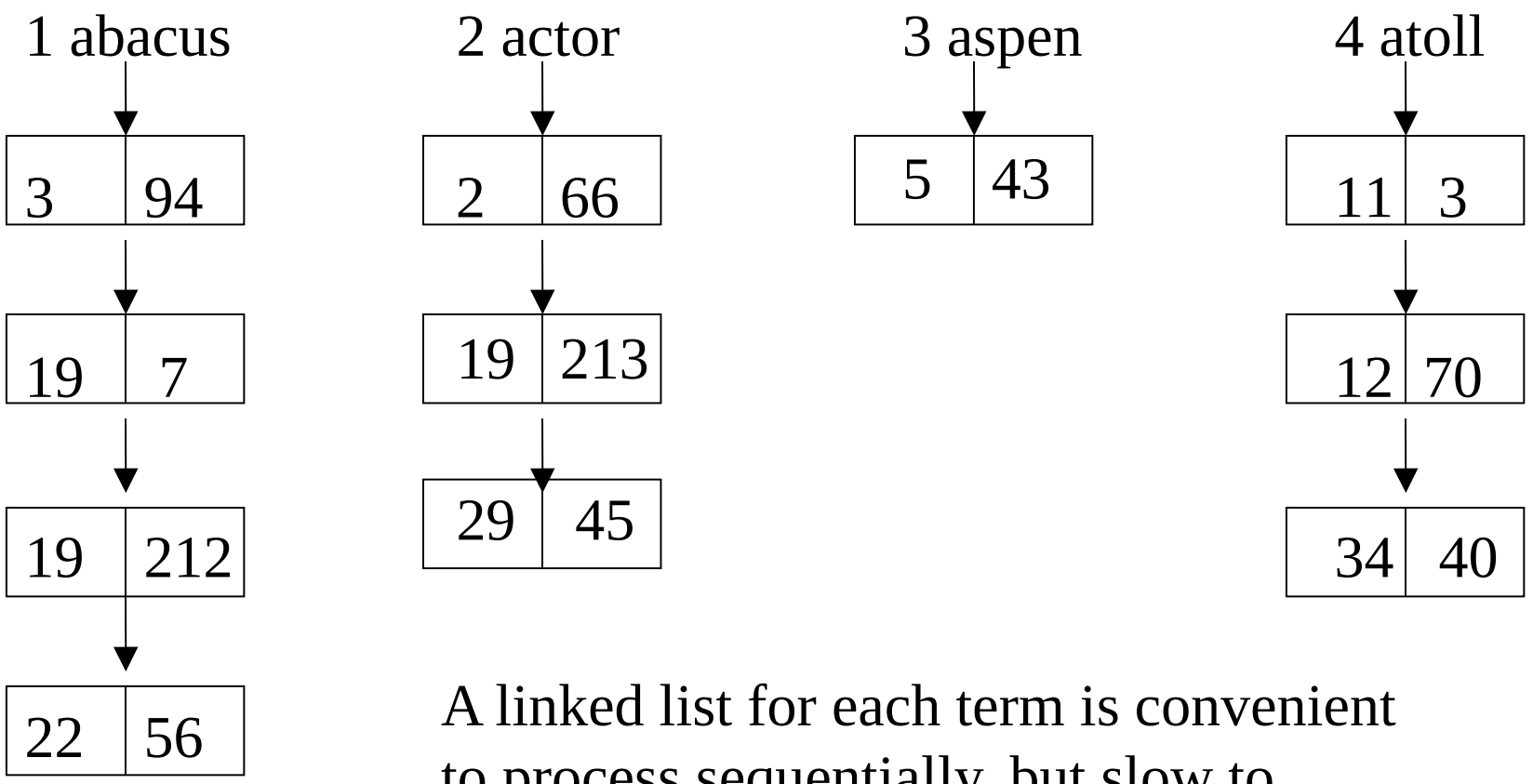
The **postings file** stores the elements of a sparse matrix, the components of the term vector space, with weights.

It is stored as a separate **inverted list** for each column, i.e., a list corresponding to each term in the index file.

Each element in an inverted list is called a **posting**, i.e., the occurrence of a term in a document

Each list consists of one or many individual postings.

Postings File: A Linked List for Each Term



A linked list for each term is convenient to process sequentially, but slow to update when the lists are long.

Length of Postings File

For a common term there may be very large numbers of postings for a given term.

Example:

1,000,000,000 documents
1,000,000 distinct words
average length 1,000 words per document

10^{12} postings

Postings File

Merging inverted lists is the most computationally intensive task in many information retrieval systems.

Since inverted lists may be long, it is important to match postings efficiently.

Usually, the inverted lists will be held on disk and paged into memory for matching. Therefore algorithms for matching postings process the lists sequentially.

For efficient matching, the inverted lists should all be sorted in the same sequence.

Inverted lists are commonly cached to minimize disk accesses.

Word List

On disk

If a word list is held on disk, search time is dominated by the number of disk accesses.

In memory

Suppose that a word list has 1,000,000 distinct terms.

Each index entry consists of the term, some basic statistics and a pointer to the inverted list, average 100 characters.

Size of index is 100 megabytes, which can easily be held in memory of a dedicated computer.